

GRENOBLE INP – ENSIMAG

2^e année – Filière ISI (Ingénierie des Systèmes d'Information)

Rapport de stage Assistant Ingénieur

Fuzzy Private Set Union avec chiffrement homomorphe de classe $\mathcal{CL}$

Intégration de la bibliothèque BICYCL dans le protocole FPSU (volePSI)

Stagiaire :

Yasser NAMATY

Tuteur :

Sylvain Bouveret

Encadrants de stage :

Aude Maignan

Jean-Guillaume Dumas

Structure d'accueil :

Laboratoire Jean Kuntzmann

(LJK)

Dates du stage : 26/05/2026 – 20/07/2026

Durée : 8 semaines

Année universitaire 2025 – 2026

Table des matières

1	Introduction	2
2	Contexte du stage	2
3	Problématique et état de l’art	3
3.1	De la PSU à la Fuzzy PSU	3
3.2	Rôle de l’OKVS et masquage homomorphe	3
3.3	Chiffrement homomorphe dans les groupes de classes	4
3.4	État de l’art et choix des outils	4
4	Méthodologie et implémentation	4
4.1	Architecture générale de la solution	4
4.2	Infrastructure logicielle et dépendances	5
4.3	Sérialisation des chiffrés dans l’OKVS	5
4.4	Plateforme de test et métriques	6
5	Résultats obtenus	6
5.1	Validation du protocole	6
5.2	Comparaison BICYCL vs Paillier	6
5.3	Coût du protocole FPSU complet	7
6	Intégration du PIR et de SimplePIR	8
6.1	Motivation : pourquoi un PIR ?	8
6.2	Découpage de l’OKVS : dense et sparse	8
6.3	SimplePIR et le pont Go/C++	9
6.4	SparseDecode1 et SparseDecode32 : état d’avancement	9
7	Analyse critique des résultats	10
8	Difficultés rencontrées	10
9	Bilan personnel	11
10	Conclusion	12
A	Guide d’installation et de compilation	13
A.1	Dépôt principal et volePSI	13
A.2	BICYCL	13
A.3	Compilation de FPSUServer_bicycl	13
A.4	RELIC (tests Paillier comparatifs) et SimplePIR	14
B	Code source de FPSUServer_bicycl.cpp	14

1 Introduction

Le partage sécurisé de données entre plusieurs parties est un enjeu central de la cryptographie appliquée : deux organisations souhaitent souvent combiner leurs informations respectives sans pour autant se dévoiler mutuellement des données confidentielles. La *Private Set Union* (PSU) formalise ce besoin : deux parties, chacune détentrice d'un ensemble d'éléments privés, cherchent à calculer l'union de leurs ensembles sans qu'aucune ne puisse apprendre les éléments propres à l'autre au-delà du résultat final. Ce problème trouve des applications directes dans le partage de données médicales, la détection collaborative de menaces réseau ou encore la publicité ciblée respectueuse de la vie privée.

Ce stage, réalisé au Laboratoire Jean Kuntzmann (LJK) au sein de l'équipe de cryptographie encadrée par Aude Maignan et Jean-Guillaume Dumas, s'inscrit dans la continuité de travaux existants sur le protocole **FPSU** (*Fuzzy Private Set Union*), une extension de la PSU qui tolère une comparaison approximative (par distance) plutôt qu'une égalité stricte entre éléments. Le protocole retenu combine un *Oblivious Key-Value Store* (OKVS, via la bibliothèque **volePSI**) et un schéma de *chiffrement homomorphe linéaire* (LHE) pour masquer les valeurs échangées.

L'objectif du stage était triple : intégrer un nouveau schéma de chiffrement homomorphe de classe $\mathcal{CL}$, via la bibliothèque **BICYCL**, au sein de l'infrastructure **volePSI** existante ; comparer ses performances à celles du schéma de Paillier, déjà disponible dans la bibliothèque **RELIC** ; et évaluer la piste d'un mécanisme de *Private Information Retrieval* (PIR), via **SimplePIR**, pour limiter la quantité d'information transférée lors de la phase de décodage. Ce rapport présente le contexte théorique nécessaire, la démarche d'implémentation suivie, les résultats expérimentaux obtenus, ainsi qu'une analyse critique du travail réalisé et des difficultés rencontrées.

2 Contexte du stage

Le Laboratoire Jean Kuntzmann (LJK) est une unité mixte de recherche associant Grenoble INP – UGA et le CNRS, structurée en plusieurs équipes couvrant les mathématiques appliquées et l'informatique. Le stage s'est déroulé au sein de l'équipe travaillant sur la cryptographie appliquée et le calcul multipartite sécurisé (*Multi-Party Computation*, MPC), sous la direction d'Aude Maignan et de Jean-Guillaume Dumas.

L'équipe développe et maintient des outils open-source de calcul sécurisé, notamment autour des primitives d'intersection et d'union d'ensembles privés (PSI/PSU). Le projet dans lequel s'inscrit ce stage vise à étendre un prototype existant de protocole FPSU afin d'y intégrer de nouveaux schémas de chiffrement homomorphe et d'en étudier les compromis performance/sécurité, dans la perspective d'une future publication ou d'un dépôt logiciel réutilisable par d'autres membres du laboratoire.

Ma mission consistait à prendre en main l'existant (le protocole FPSU basé sur **volePSI**), à y intégrer la bibliothèque **BICYCL** pour le chiffrement homomorphe de classe $\mathcal{CL}$, à mener une campagne de benchmarks comparatifs avec le schéma de Paillier, puis à explorer l'ajout d'un mécanisme PIR pour réduire le coût de communication du protocole.

3 Problématique et état de l’art

3.1 De la PSU à la Fuzzy PSU

Définition 3.1 (Private Set Union). *Deux parties A et B possèdent chacune un ensemble privé d’éléments. La PSU consiste à calculer $A \cup B$ de telle sorte qu’aucune des deux parties n’apprenne d’information sur les éléments de l’autre autres que ceux figurant dans l’union finale.*

La PSU classique ne fusionne que des éléments strictement identiques. Or, dans de nombreuses applications réelles (données biométriques, coordonnées GPS, séquences ADN, chaînes de texte), les données sont naturellement bruitées : deux mesures d’un même objet ne coïncident presque jamais bit à bit. La **Fuzzy PSU** remplace donc le test d’égalité stricte par une comparaison de distance tolérant un rayon δ : deux éléments x et y sont considérés comme identiques si $\|x - y\| \leq \delta$.

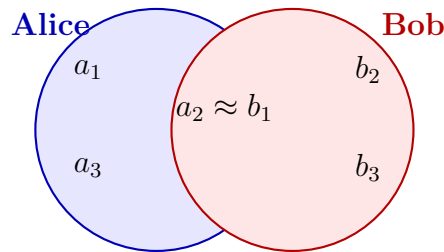


FIGURE 1 – Principe de la Fuzzy PSU : les éléments a_2 et b_1 , suffisamment proches (à distance inférieure au seuil δ), sont fusionnés en un seul élément dans l’union finale $\{a_1, a_2, a_3, b_2, b_3\}$.

Pour rendre ce test de proximité efficace sans comparer chaque élément de A à chaque élément de B (coût quadratique), chaque élément est représenté par une *boule* de rayon δ autour de lui, elle-même projetée sur chaque dimension : une projection génère $2\delta + 1$ clés candidates. C’est cet ensemble de clés qui est ensuite inséré dans une structure d’*Oblivious Key-Value Store* (OKVS), implémentée ici par l’algorithme Paxos de la bibliothèque volePSI.

3.2 Rôle de l’OKVS et masquage homomorphe

L’OKVS permet à chaque partie d’encoder ses paires (clé, valeur) de façon à ce que la valeur associée à une clé donnée puisse être retrouvée par un décodage, sans que les clés utilisées lors de l’encodage ne soient révélées. Dans notre protocole, chaque clé (une projection) est associée à une valeur aléatoire $r_{i,k}$, telle que la somme des parts sur les deux serveurs vaut zéro lorsque les clés correspondent réellement :

$$\sum_{k=1}^d r_{i,k} = 0 \quad \text{si et seulement si les projections coïncident.}$$

Vérifier cette égalité sans révéler $r_{i,1}$ ni $r_{i,2}$ nécessite un mécanisme qui permette de combiner des valeurs chiffrées sans les déchiffrer : un **chiffrement homomorphe additif**. C’est ce rôle que joue le schéma $\mathcal{CL}$ -HSMqk implémenté par BICYCL dans notre protocole FPSUServer_bicycl : les parts sont chiffrées avant leur insertion dans l’OKVS, l’addition

homomorphe $E(r_{i,1}) \oplus E(r_{i,2}) = E(r_{i,1} + r_{i,2})$ est effectuée sur les chiffrés, et une unique opération de déchiffrement final suffit à vérifier la correspondance.

3.3 Chiffrement homomorphe dans les groupes de classes

Le chiffrement $\mathcal{CL}$ repose sur la difficulté de certains problèmes dans les groupes de classes d'idéaux de corps quadratiques imaginaires.

Définition 3.2 (Groupe de classes). *Soit $\Delta_K < 0$ un discriminant fondamental. Le groupe de classes d'idéaux $\text{Cl}(\Delta_K)$ est le groupe quotient des idéaux fractionnaires par les idéaux principaux de l'ordre $\mathcal{O}_K$.*

Le schéma $\mathcal{CL}$ -HSMqk [4], variante du schéma de Castagnos-Laguillaumie, exploite deux sous-groupes emboîtés du groupe de classes : un sous-groupe d'ordre connu et premier q , qui sert à encoder le message via un générateur f , et le reste du groupe, d'ordre inconnu, qui sert à masquer le message (aléa de chiffrement). Le schéma se décrit ainsi :

1. **KeyGen** : tirer une clé secrète x , publier $pk = g^x$;
2. **Encrypt**(m) : tirer un aléa r , calculer $c_1 = g^r$ et $c_2 = pk^r \cdot f^m$ (composition de formes quadratiques, via NUCOMP) ;
3. **Decrypt** : retirer le masque grâce à x , puis retrouver m en résolvant un logarithme discret dans le sous-groupe d'ordre q , connu et petit, donc facile à calculer ;
4. **Add** : composer directement les formes quadratiques des deux chiffrés, ce qui donne le chiffré de la somme.

3.4 État de l'art et choix des outils

Le protocole s'appuie sur **volePSI** [1], bibliothèque C++ de Visa Research qui fournit les primitives *Vector Oblivious Linear Evaluation* (VOLE), *Oblivious Transfer* (OT) et la structure OKVS Paxos. Pour le chiffrement homomorphe, deux options ont été comparées : les schémas Paillier [6], disponibles via la bibliothèque **RELIC**, et le schéma $\mathcal{CL}$ -HSMqk, disponible via **BICYCL** [3], bibliothèque C++20 développée par l'équipe CRYPTO du LIRMM. BICYCL a été retenue pour l'implémentation principale car, à niveau de sécurité équivalent, ses chiffrés sont plus compacts (module de 256 bits contre des modules RSA de 3072 bits pour Paillier), tout en offrant une interface C++ moderne facile à intégrer dans la chaîne de compilation de **volePSI**. Enfin, pour la piste PIR, **SimplePIR** [7], implémentée en Go, a été retenue pour sa simplicité d'intégration via un pont **cgo**.

4 Méthodologie et implémentation

4.1 Architecture générale de la solution

L'architecture retenue s'articule en trois couches, illustrées en figure 2 : le protocole **FPSU** orchestre l'ensemble ; l'**OKVS** (Paxos, dans **volePSI**) stocke des valeurs qui sont désormais des *chiffrés* BICYCL, décomposé en une partie *dense* (petite, transmise directement) et une partie *sparse* (grande, candidate à une interrogation privée via PIR) ; enfin, le **chiffrement homomorphe** BICYCL assure la confidentialité des valeurs partagées tout au long du calcul.

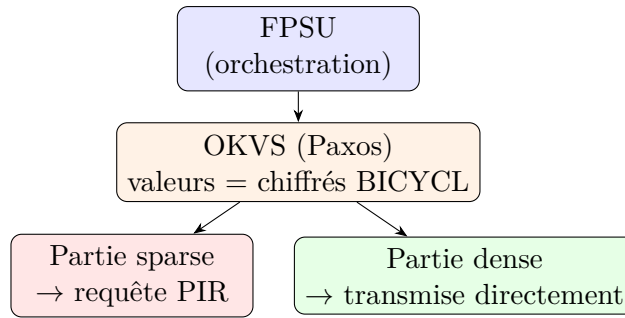


FIGURE 2 – Architecture logicielle de `FPSUServer_bicycl` : le protocole FPSU s’appuie sur un OKVS dont les valeurs sont chiffrées avec BICYCL ; la partie *sparse* de la structure, la plus volumineuse, est la cible envisagée pour une interrogation privée (PIR).

4.2 Infrastructure logicielle et dépendances

La mise en œuvre a nécessité l’articulation de quatre bibliothèques externes, chacune avec ses propres contraintes de compilation :

- **volePSI** [1] : couche cryptographique centrale, compilée avec `clang++-16` ou `g++ ≥ 13` (norme C++20 requise) ;
- **BICYCL** [3] : implémentation du schéma $\mathcal{CL}$ -HSMqk, compilée séparément via CMake pour ses propres tests puis liée statiquement à `FPSUServer_bicycl` ;
- **RELIC** [5] : utilisée uniquement pour les tests comparatifs Paillier (SHPE/PHPE) ;
- **SimplePIR** [7] : implémentation Go du protocole PIR, exposée en C via un pont `cgo` compilé en bibliothèque partagée (`libsimplepir.so`).

La compilation de `FPSUServer_bicycl` combine ainsi des symboles provenant de quatre chaînes de compilation distinctes (C++20 pour `volePSI`, CMake/C++20 pour `BICYCL`, C pour `RELIC`, Go pour `SimplePIR`), ce qui a constitué l’une des principales sources de difficultés techniques du stage (voir section 8).

4.3 Sérialisation des chiffrés dans l’OKVS

La structure `Paxos` de `volePSI` manipule des valeurs de taille fixe (`ValueType`), alors que les chiffrés $\mathcal{CL}$ -HSMqk sont des paires de formes quadratiques de taille variable. J’ai donc défini `ValueType` comme un tableau fixe de blocs de 128 bits (4096 octets, soit une marge confortable au-dessus des 4663 bits mesurés pour un chiffré à 128 bits de sécurité), et implémenté :

- une sérialisation des entiers signés (`Mpz`) de BICYCL sous forme d’un octet de signe suivi d’une longueur et des octets du nombre ;
- un encodage des formes quadratiques (`QFI`) à partir de leurs trois coefficients (a, b, c) ;
- les opérateurs $\wedge$ (XOR) et $\&$ requis par `Paxos` sur le type `ValueType`, appliqués bloc par bloc.

Le code complet de `FPSUServer_bicycl.cpp`, comprenant cette couche de sérialisation, le chiffrement des parts partagées, l’encodage/décodage Paxos et la reconstruction homomorphe, est fourni en annexe B.

4.4 Plateforme de test et métriques

Les tests ont été menés sur un serveur du laboratoire (LJK-AD-200), pour un protocole avec $n = 8$ éléments par ensemble, un rayon de tolérance $\delta = 2$ (soit $N = (2\delta + 1)n = 40$ clés encodées) et un niveau de sécurité de 128 bits. Les métriques suivies sont : le temps d’encodage/décodage Paxos, le temps des opérations cryptographiques élémentaires (génération de clé, chiffrement, déchiffrement, addition homomorphe) et le volume de communication échangé. Ces mesures ont été comparées, à niveau de sécurité identique, entre BICYCL et les deux variantes de Paillier disponibles dans RELIC (SHPE et PHPE).

5 Résultats obtenus

5.1 Validation du protocole

Le décodage de l’OKVS a été testé sur cinq requêtes ($m = 5$) : deux couples de clés synchronisées entre les deux serveurs, un couple de clés désynchronisées, et deux couples de clés tirées aléatoirement (absentes de la structure). Les résultats confirment le bon fonctionnement du mécanisme : les correspondances réelles sont systématiquement retrouvées, et aucune fausse acceptation n’a été observée sur les clés désynchronisées ou aléatoires, ce qui valide le comportement attendu de l’addition homomorphe couplée au décodage Paxos.

5.2 Comparaison BICYCL vs Paillier

Le tableau 1 synthétise les performances mesurées à niveau de sécurité équivalent (128 bits), pour des messages de 256 bits.

TABLE 1 – Comparaison BICYCL vs Paillier (SecLevel 128 bits)

Critère	BICYCL	Paillier	Gain
Taille des chiffrés	4 663 bits	6 144 bits	BICYCL (−24%)
Génération de clés	24.45 ms	183 – 1353 ms	BICYCL (7–55×)
Chiffrement	9.44 ms	85 – 164 ms	BICYCL (9–17×)
Déchiffrement	23.58 ms	3.7 – 22 ms	Paillier
Addition homomorphe	11 073 μ s	70 μ s	Paillier ($\sim$ 158×)

BICYCL produit des chiffrés 24% plus compacts et un chiffrement nettement plus rapide, au prix d’une addition homomorphe et d’un déchiffrement plus coûteux. Ce compromis est favorable pour FPSU, où l’opération dominante est le chiffrement des valeurs partagées lors de la construction de l’OKVS, et où la compacité des chiffrés réduit directement le volume de communication – un critère particulièrement important lorsque le nombre de clients augmente.

Pour compléter cette analyse, le tableau 2 détaille les performances de BICYCL pour les trois niveaux de sécurité testés (112, 128 et 192 bits), tous avec un module q de 256 bits. Ces mesures confirment que le coût des opérations croît de façon sensiblement plus que linéaire avec le niveau de sécurité visé – passer de 128 à 192 bits multiplie le temps de chiffrement par environ 3,9 et celui de l’addition homomorphe par 3,2 – ce qui doit être pris en compte lors du choix des paramètres pour un déploiement réel.

TABLE 2 – Temps d'exécution de BICYCL selon le niveau de sécurité ($q = 256$ bits)

SecLevel	KGen(pk)	Encrypt	Decrypt	Add(ct)	Dec(sum)
112 bits	14.38 ms	6.22 ms	12.95 ms	5 504 μ s	13.74 ms
128 bits	24.45 ms	9.44 ms	23.58 ms	11 073 μ s	22.14 ms
192 bits	95.51 ms	36.54 ms	87.49 ms	35 569 μ s	88.99 ms

Du côté de RELIC, deux variantes de Paillier ont en réalité été testées : **PHPE** (Paillier standard) et **SHPE** (variante *sous-groupe* de Paillier, à module réduit **sbits**). Le tableau 3 compare les deux variantes à 128 bits de sécurité, pour des messages de 256 bits : SHPE se distingue par un déchiffrement et une addition homomorphe nettement plus rapides que PHPE, au prix d'une génération de clé beaucoup plus coûteuse (1353 ms contre 184 ms). Ce résultat illustre qu'au sein même de la famille Paillier, le choix des paramètres déplace déjà sensiblement le compromis performance ; la comparaison avec BICYCL, menée plus haut avec PHPE comme référence usuelle, resterait donc à affiner face à SHPE dans un travail de suivi.

TABLE 3 – PHPE vs SHPE (SecLevel 128 bits, message 256 bits)

Critère	PHPE	SHPE	Écart
Génération de clés	183.76 ms	1 353.04 ms	SHPE 7,4× plus lent
Chiffrement	84.75 ms	163.63 ms	PHPE 1,9× plus rapide
Déchiffrement	22.36 ms	3.74 ms	SHPE 6× plus rapide
Addition homomorphe	0.07 ms	0.07 ms	équivalent

5.3 Coût du protocole FPSU complet

Le tableau 4 compare la version de base de **FPSUServer** (sans chiffrement) à **FPSUServer_bicycl**, pour $n = 8$.

TABLE 4 – Temps d'exécution et communication pour $n = 8$

Variante	Temps total	Envoyé	Reçu
FPSUServer (base)	0.134 ms	0.5 KB	0.5 KB
FPSUServer_bicycl	33.70 ms	2.3 KB	2.3 KB

Le surcoût en temps (facteur ~ 250) s'explique par les $2n$ chiffrements nécessaires pour les parts partagées et par la sérialisation/désérialisation des chiffrés dans la structure Paxos. Le surcoût en communication (facteur ~ 4.6) correspond directement au rapport entre la taille d'un chiffré BICYCL (4663 bits) et celle d'une valeur en clair (128 bits). Ce surcoût, significatif en valeur relative, reste acceptable en valeur absolue (quelques dizaines de millisecondes) pour des ensembles de taille modérée ($n \leq 100$), et constitue le prix à payer pour la confidentialité homomorphe des valeurs échangées.

6 Intégration du PIR et de SimplePIR

Au-delà du chiffrement homomorphe, la seconde piste d’optimisation étudiée pendant le stage concerne le coût de *communication* du protocole FPSU lors de la phase de décodage côté client. Cette section détaille la motivation, l’architecture envisagée et l’état d’avancement de cette intégration, qui a occupé une part significative de la fin du stage.

6.1 Motivation : pourquoi un PIR ?

Dans le protocole FPSU tel qu’implémenté jusqu’ici, un client qui souhaite tester si un élément lui appartenant figure dans la structure OKVS du serveur doit, pour décoder une clé, accéder à plusieurs positions de la structure **storage** produite par Paxos. Deux stratégies extrêmes sont possibles : soit le client télécharge l’intégralité de **storage** (ce qui garantit la confidentialité de la position recherchée, mais coûte cher en communication dès que n grandit), soit il demande explicitement les indices dont il a besoin (ce qui est peu coûteux, mais révèle immédiatement au serveur quels éléments il recherche, ruinant la confidentialité recherchée par la PSU). Le *Private Information Retrieval* (PIR) permet de sortir de ce dilemme : il autorise un client à récupérer la valeur stockée à une position donnée d’une base de données, sans que le serveur n’apprenne quelle position a été interrogée.

6.2 Découpage de l’OKVS : dense et sparse

L’analyse de la structure produite par Paxos montre qu’elle se décompose naturellement en deux parties de tailles très différentes, illustrées en figure 3 :

- une partie **dense**, de taille $\mathcal{O}(\log n)$, suffisamment petite pour être transmise directement au client sans risque significatif pour la confidentialité ni pour la communication ;
- une partie **sparse**, de taille $\mathcal{O}(n)$, nettement plus volumineuse, et donc la cible naturelle d’une interrogation privée.

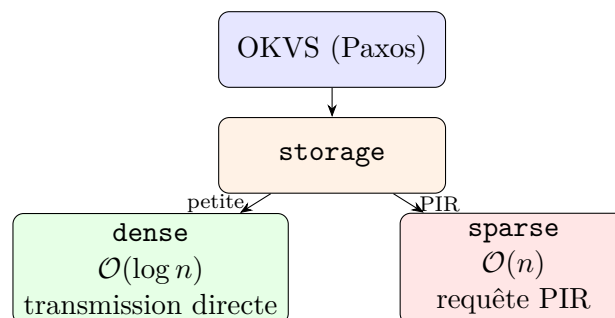


FIGURE 3 – Découpage de la structure OKVS produite par Paxos : la partie **dense** est transmise directement, tandis que la partie **sparse**, nettement plus volumineuse, est destinée à une interrogation privée via SimplePIR. Seuls les quelques bits correspondant au hachage de la clé recherchée sont nécessaires côté client.

6.3 SimplePIR et le pont Go/C++

SimplePIR [7] est un protocole de PIR mono-serveur fondé sur la difficulté du problème *Learning With Errors* (LWE) : le serveur organise sa base de données sous forme de matrice, le client envoie une requête chiffrée (un vecteur creux masqué par du bruit LWE), le serveur répond par un produit matrice-vecteur effectué directement sur les données chiffrées, et le client retrouve l'élément recherché par déchiffrement LWE. Son intérêt principal pour notre cas d'usage est sa simplicité relative d'implémentation et son efficacité pour des bases de données de taille modérée.

SimplePIR étant distribué en Go, son intégration dans la chaîne de compilation C++ de volePSI a nécessité la construction d'un pont via `cgo`, exposant quatre primitives sous forme de bibliothèque partagée (`libsimplepir.so`) :

```

1 //export GlobalInitPIR
2 func GlobalInitPIR()
3
4 //export PIRServerSetup
5 func PIRServerSetup(serverId int, size uint64)
6 // construit la base PIR (matrice ~sqrt(2n) x sqrt(2n))
7
8 //export PIRClientQuery
9 func PIRClientQuery(targetIndex int)
10 // genere une requete chiffrée pour l'index recherche
11
12 //export PIRServerAnswer
13 func PIRServerAnswer(serverId int)
14 // multiplication matrice-vecteur sur la base chiffrée
15
16 //export PIRClientExtract
17 func PIRClientExtract()
18 // déchiffrement LWE cote client, extraction de l'element

```

Listing 1 – Extrait de `bridge.go` – interface exposée au code C++

Ces quatre fonctions ont été testées isolément avec succès (démonstration intégrée dans `FPSUServer_bicycl`, voir annexe B) : elles constituent la brique de base sur laquelle s'appuiera l'intégration complète au décodage de l'OKVS.

6.4 SparseDecode1 et SparseDecode32 : état d'avancement

L'objectif final est de remplacer, dans la phase de décodage côté client, la lecture directe de la partie `sparse` du `storage` par une (ou plusieurs) requête(s) PIR. Deux variantes ont été identifiées avec mes encadrants :

- **SparseDecode1** : chaque décodage nécessite la récupération d'une seule position de la partie `sparse` (cas le plus simple, une seule requête PIR par décodage) ;
- **SparseDecode32** : le décodage nécessite de combiner (par XOR) jusqu'à 32 positions de la partie `sparse`, ce qui correspond au paramétrage réel utilisé par Paxos pour garantir un taux d'échec négligeable de l'encodage OKVS. Cette variante est plus représentative du fonctionnement réel du protocole, mais complique l'intégration du PIR : il faut soit émettre 32 requêtes PIR indépendantes (coût de communication multiplié par 32), soit concevoir une requête PIR « groupée » qui récupère les 32 positions en une seule interaction avec le serveur.

À l'issue du stage, **SparseDecode1 avec PIR** a été implémenté et testé avec succès, aussi bien dans sa version sans PIR (lecture directe, servant de référence) que dans sa version utilisant SimplePIR. Le passage direct à **SparseDecode32**, entrepris dans la foulée sans repasser par une version intermédiaire sans PIR, s'est révélé plus complexe

que prévu : la méthode à suivre (agrégation homomorphe-compatible des 32 réponses PIR) est désormais bien comprise, mais son implémentation n'a pas pu être finalisée dans le temps imparti du stage. Cette étape figure explicitement dans les perspectives de poursuite du projet (section 10).

7 Analyse critique des résultats

Les objectifs fixés en introduction ont été largement atteints. L'intégration de BICYCL dans volePSI fonctionne et a été validée expérimentalement ; la campagne de benchmarks a permis de dégager un compromis clair entre BICYCL et Paillier, orientant le choix vers BICYCL pour ce protocole précis. Le volet PIR a également progressé au-delà du simple pont logiciel : la variante `SparseDecode1` fonctionne de bout en bout, avec et sans PIR, ce qui valide le principe d'une interrogation privée de la partie *sparse* de l'OKVS. Seule la variante `SparseDecode32`, plus représentative du paramétrage réel de Paxos, reste à finaliser (section 6).

Sur le plan de la méthode, les tests ont été menés sur des ensembles de petite taille ($n = 8$), ce qui limite la portée des conclusions de scalabilité : une estimation par extrapolation linéaire suggère un temps total de l'ordre de 420 ms pour $n = 100$, mais cette estimation reste à confirmer expérimentalement. De même, le choix des paramètres de sécurité (module q de 256 bits) et l'absence de chiffrement par lots (*batching*) laissent une marge d'optimisation notable, en particulier pour réduire le coût de l'addition homomorphe, point faible identifié de BICYCL face à Paillier.

8 Difficultés rencontrées

Ce stage a représenté ma première expérience de développement sur un projet de cryptographie appliquée de cette ampleur, et plusieurs types de difficultés, techniques comme conceptuelles, ont jalonné le travail.

Installation et compatibilité des bibliothèques. Le projet repose sur quatre bibliothèques externes (volePSI, BICYCL, RELIC, SimplePIR), chacune avec ses propres exigences de compilateur et de version. volePSI nécessite un compilateur récent supportant pleinement le C++20 (`clang++-16` ou `g++ ≥ 13`), alors que l'environnement de départ ne disposait pas d'un compilateur suffisamment récent, ce qui a nécessité d'installer et de configurer `clang++-16` en parallèle du compilateur système. Ce type de problème, en apparence anodin, s'est révélé chronophage : une simple divergence de version de compilateur provoque des erreurs de compilation difficiles à diagnostiquer, car les messages d'erreur pointent rarement directement vers la cause réelle (incompatibilité de standard du langage plutôt qu'erreur de syntaxe).

Problèmes de liaison (*linking*). Au-delà de la compilation, l'édition de liens s'est révélée être une source récurrente d'erreurs : l'ordre des bibliothèques passées à l'éditeur de liens (`-lvolePSI -llib0Te -lcryptoTools`, etc.) doit respecter les dépendances entre symboles, sous peine d'erreurs `undefined reference` difficiles à interpréter pour un projet de cette taille. Un cas particulier a nécessité une attention systématique : la compilation des tests BICYCL modifie le `CMakeLists.txt` du projet, et un oubli de restauration de ce fichier avant de recompiler `FPSUServer_bicycl` provoque un échec de

l'éditeur de liens (`linker command failed with exit code 1`) sans lien évident avec la cause réelle. Identifier ce type de dépendance cachée entre étapes de compilation demande une démarche de débogage méthodique, par élimination.

Maîtrise des notions théoriques. La compréhension fine du schéma $\mathcal{CL}$ -HSMqk (groupes de classes d'idéaux, sous-groupes emboîtés d'ordre connu/inconnu, composition de formes quadratiques via NUCOMP) a demandé un travail de lecture et d'appropriation substantiel, la cryptographie sur les groupes de classes étant nettement moins répandue dans la littérature pédagogique que les schémas RSA ou Paillier. Il a fallu également comprendre en profondeur l'articulation entre VOLE, OKVS et chiffrement homomorphe au sein du protocole FPSU pour pouvoir modifier correctement le code existant sans en briser les propriétés de sécurité – une erreur de conception à ce niveau (par exemple un mauvais encodage des parts partagées) aurait pu invalider silencieusement les garanties du protocole sans provoquer d'erreur visible à l'exécution.

Compréhension minutieuse du sujet. Le sujet du stage a évolué progressivement à mesure que la compréhension du problème s'affinait : la distinction entre PSU et Fuzzy PSU, le rôle exact de chaque sous-composant de l'OKVS (parties *dense* et *sparse*), et la façon dont le PIR devait s'y greffer n'étaient pas évidents dès le départ. Cela a nécessité des échanges réguliers avec mes encadrants pour ajuster ma compréhension et éviter de partir dans une direction d'implémentation inadaptée.

Rédaction en C++. Enfin, la rédaction du code elle-même a représenté une difficulté à part entière : la sérialisation de structures cryptographiques de taille variable (les formes quadratiques QFI de BICYCL) dans un type de taille fixe imposé par `Paxos` a demandé une gestion rigoureuse de la mémoire (tampons, tailles, cas limites), un domaine dans lequel je manquais initialement d'expérience pratique. Le pont entre le monde Go de SimplePIR et le monde C++ de volePSI, via `cgo`, a ajouté une difficulté supplémentaire de gestion d'interfaces entre langages.

Ces difficultés, bien que parfois frustrantes sur le moment, ont constitué une part importante de l'apprentissage du stage : elles m'ont conduit à développer une méthode de débogage plus systématique et une meilleure compréhension des chaînes de compilation C++ complexes.

9 Bilan personnel

Ce stage m'a permis de consolider des compétences techniques en cryptographie appliquée (chiffrement homomorphe, protocoles multipartites sécurisés) que je n'avais jusqu'à là abordées que de manière théorique. J'ai également renforcé mes compétences en C++ moderne (C++20, gestion de templates, interopérabilité avec des bibliothèques externes) ainsi qu'en intégration de systèmes hétérogènes (C++, C, Go).

Le travail mené sur le PIR et SimplePIR a été, à titre personnel, l'une des parties les plus formatrices du stage : c'est la première fois que j'ai dû faire coopérer deux écosystèmes de langages aussi différents que Go et C++20 au sein d'un même exécutable. Comprendre le fonctionnement de `cgo`, gérer la compilation d'une bibliothèque partagée exportant des symboles C depuis du Go, puis les lier correctement à volePSI, m'a demandé de sortir de ma zone de confort habituelle (plutôt orientée C++ et Python). Cela m'a aussi appris à

découper un problème complexe (l'intégration complète du PIR au décodage OKVS) en étapes vérifiables indépendamment : valider d'abord les quatre primitives PIR isolément, puis les brancher sur le cas le plus simple (`SparseDecode1`) avant de m'attaquer au cas général (`SparseDecode32`). Cette démarche incrémentale, bien qu'elle ne m'ait pas permis de finaliser entièrement l'intégration dans le temps du stage, m'a évité de me retrouver bloqué sur un problème monolithique impossible à déboguer.

Sur le plan des compétences interpersonnelles, j'ai appris à communiquer régulièrement l'avancement de travaux exploratoires, dont les résultats ne sont pas toujours garantis à l'avance, et à solliciter mes encadrants au bon moment plutôt que de rester bloqué seul sur un problème. La rigueur exigée par la cryptographie – où une erreur d'implémentation peut rester invisible sans compromettre silencieusement la sécurité du protocole – m'a également habitué à une démarche de validation plus systématique (tests de cas positifs et négatifs, vérifications croisées) que je compte conserver dans mes futurs projets.

10 Conclusion

Ce stage avait pour objectif d'intégrer le chiffrement homomorphe de classe $\mathcal{CL}$, via la bibliothèque BICYCL, au sein du protocole FPSU implémenté sur `volePSI`, de comparer ses performances à celles du schéma de Paillier, et d'explorer une piste d'optimisation du coût de communication par PIR. Le premier objectif a été atteint : `FPSU_Server_bicycl` fonctionne et a été validé sur des tests de décodage positifs et négatifs. Le second objectif a également été atteint, avec une campagne de benchmarks complète montrant que BICYCL offre des chiffrés 24% plus compacts et un chiffrement jusqu'à 17 fois plus rapide que Paillier, au prix d'une addition homomorphe environ 158 fois plus lente – un compromis favorable pour ce protocole, où le chiffrement domine le coût global. Le troisième objectif, l'intégration du PIR, a nettement progressé : au-delà du pont logiciel Go/C++ reliant SimplePIR à `volePSI`, la variante `SparseDecode1` a été intégrée et validée de bout en bout, avec et sans PIR. Seule la variante `SparseDecode32`, plus proche du paramétrage réel du protocole, reste à finaliser.

Les perspectives de ce travail, à poursuivre dans la suite du projet au LJK, incluent : la finalisation de `SparseDecode32` avec SimplePIR, puis l'intégration de la version complète combinant chiffrement BICYCL et PIR au sein d'un même exécutable ; le passage à l'échelle sur des ensembles de taille réaliste ($n \geq 10^3$) ; et l'exploration du chiffrement par lots (*batching*) pour réduire le coût de l'addition homomorphe de BICYCL, principal point faible identifié face à Paillier.

Références bibliographiques

Références

- [1] P. Rindal, *volePSI : VOLE-based Private Set Intersection/Union*, <https://github.com/Visa-Research/volepsi>, 2022.
- [2] Jean-Guillaume Dumas, Aude Maignan, Luiza Soezima, *Fuzzy Private Set Union via Oblivious Key Homomorphic Encryption Retrieval*, HAL Id : hal-05477627, <https://hal.science/hal-05477627v1>

- [3] C. Bouvier, L. Imbert, T. Prest, *BICYCL : A C++ library for cryptography in class groups*, LIRMM / Inria, <https://gite.lirmm.fr/crypto/bicycl>, 2022.
- [4] C. Castagnos, F. Laguillaumie, *Linearly Homomorphic Encryption from DDH*, CT-RSA 2015, LNCS 9048, pp. 487–505, 2015.
- [5] D. Aranha, C. Gouvêa, *RELIC is an Efficient Library for Cryptography*, <https://github.com/relic-toolkit/relic>.
- [6] P. Paillier, *Public-Key Cryptosystems Based on Composite Degree Residuosity Classes*, EUROCRYPT 1999, LNCS 1592, pp. 223–238, 1999.
- [7] A. Henzinger, M. Hong, H. Corrigan-Gibbs, S. Meiklejohn, V. Vaikuntanathan, *One Server for the Price of Two : Simple and Fast Single-Server Private Information Retrieval*, USENIX Security 2023, <https://github.com/ahenzinger/simplepir>.

A Guide d’installation et de compilation

Cette annexe reprend, à titre de documentation technique, les étapes nécessaires pour reproduire l’environnement de compilation utilisé pendant le stage.

A.1 Dépôt principal et volePSI

```

1 git clone https://github.com/yasserito111/CRYPTO.git
2 git clone https://github.com/Visa-Research/volepsi.git
3 cd volepsi
4 python3 build.py -DVOLE_PSI_ENABLE_BOOST=ON
5 # Si le compilateur par défaut est insuffisant (C++20 requis) :
6 sudo apt install clang++-16

```

Listing 2 – Clonage et compilation de volePSI

A.2 BICYCL

```

1 git clone https://gite.lirmm.fr/crypto/bicycl.git
2 mv CRYPTO/test_CL_HSMqk_128.cpp bicycl/tests/
3 # Ajouter "test_CL_HSMqk_128" a TESTS_LIST dans bicycl/tests/CMakeLists.txt
4 cd bicycl && rm -rf build
5 cmake -S . -B build
6 cmake --build build --target test_CL_HSMqk_128 -j
7 ./build/tests/test_CL_HSMqk_128
8 # IMPORTANT : restaurer CMakeLists.txt (retirer la ligne ajoutée) puis
9 # rm -rf build && cmake -S . -B build avant de compiler FPSUServer_bicycl,
10 # sous peine d'erreur de l'éditeur de liens.

```

Listing 3 – Clonage de BICYCL et test CL_HSMqk_128

A.3 Compilation de FPSUServer_bicycl

```

1 cd volepsi
2 mv ../CRYPTO/FPSUServer_bicycl.cpp .
3 clang++-16 FPSUServer_bicycl.cpp \
4     -std=c++20 -maes -mpclmul -mavx2 \
5     -I. -I./out/install/linux/include -I./volePSI \
6     -I$HOME/bicycl/src \
7     -L./build/volePSI -L./out/install/linux/lib \
8     -o FPSUServer_bicycl \
9     -Wl,--start-group \

```

```

10      -lvolePSI -llibOTe -lbitpolymul \
11      -lcoprotos -lmacoro -lcryptoTools \
12      -Wl,--end-group \
13      -lsodium -lgmp -lgmpxx -lssl -lcrypto -lpthread
14  ./FPSUServer_bicycl

```

Listing 4 – Compilation finale

A.4 RELIC (tests Paillier comparatifs) et SimplePIR

```

1  git clone https://github.com/relic-toolkit/relic.git
2  # ... compilation des tests SHPE/PHPE (voir depot du projet) ...
3
4  git clone https://github.com/ahenzinger/simplepir.git
5  cd simplepir
6  go mod tidy
7  go build -buildmode=c-shared -o libsimplepir.so bridge.go
8  cp libsimplepir.so libsimplepir.h ~/volepsi/

```

Listing 5 – RELIC et pont SimplePIR

B Code source de FPSUServer_bicycl.cpp

Le code ci-dessous présente la dernière version du serveur FPSU intégrant BICYCL, telle qu’obtenue à l’issue du stage. Par rapport à la version initiale, elle apporte trois évolutions notables : (i) un module q de chiffrement passé d’un petit entier de test (65537) à un nombre premier de 256 bits (dimensionnement réaliste pour un niveau de sécurité de 128 bits), (ii) un préfixe de longueur sur 16 bits (u16) au lieu de 32 bits pour la sérialisation des coefficients `Mpz`, ce qui suffit largement pour des coefficients de quelques centaines de bits et réduit légèrement la taille du tampon sérialisé, et (iii) l’ajout de fonctions de journalisation (`debug_mpz`, messages de diagnostic sur la taille des chiffrés) qui ont grandement facilité le débogage évoqué en section 8. La capacité du type `ValueType` est fixée à 38 blocs de 128 bits (608 octets, soit 4864 bits), une marge volontairement confortable au-dessus des 4663 bits mesurés pour un chiffré BICYCL à 128 bits de sécurité.

```

1  #include "cryptoTools/Common/Defines.h"
2  #include "cryptoTools/Crypto/PRNG.h"
3  #include <bicycl.hpp>
4
5  #include <iostream>
6  #include <vector>
7  #include <chrono>
8  #include <string>
9  #include <array>
10 #include <sstream>
11 #include <cstring>
12 #include <cstdint>
13 #include <algorithm>
14
15 using namespace osuCrypto;
16
17 constexpr size_t kMaxValueBlocks = 38;
18 constexpr size_t kMaxValueBytes = kMaxValueBlocks * sizeof(block);
19 using ValueType = std::array<block, kMaxValueBlocks>;
20
21 namespace osuCrypto {
22     using ValueTypeAlias = ::ValueType;
23     ValueTypeAlias operator^(const ValueTypeAlias& a, const ValueTypeAlias& b);
24     ValueTypeAlias operator&(const ValueTypeAlias& a, const ValueTypeAlias& b);
25 }
26

```

```

27 #include "volePSI/Paxos.h"
28
29 // -----
30 // Operateurs requis par Paxos sur ValueType
31 // -----
32 namespace osuCrypto {
33     ValueType operator^(const ValueType& a, const ValueType& b) {
34         ValueType result;
35         for (size_t i = 0; i < result.size(); ++i)
36             result[i] = a[i] ^ b[i];
37         return result;
38     }
39
40     ValueType& operator^=(ValueType& a, const ValueType& b) {
41         for (size_t i = 0; i < a.size(); ++i)
42             a[i] = a[i] ^ b[i];
43         return a;
44     }
45
46     ValueType operator&(const ValueType& a, const ValueType& b) {
47         ValueType result;
48         for (size_t i = 0; i < result.size(); ++i)
49             result[i] = a[i] & b[i];
50         return result;
51     }
52 }
53
54 std::ostream& operator<<(std::ostream& os, const ValueType& v) {
55     for (const auto& blk : v)
56         os << blk << " ";
57     return os;
58 }
59
60 // Fonction de journalisation ajoutée pour le debogage (section 7 du rapport)
61 static void debug_mpz(const char* label, const BICYCL::Mpz& value) {
62     std::vector<unsigned char> bytes = static_cast<std::vector<unsigned char>>(value);
63     std::cout << "[debug] " << label
64               << " nbits=" << value.nbits()
65               << " abs_bytes=" << bytes.size()
66               << " sign=" << value.sgn() << "\n";
67 }
68
69 bool is_zero_valuetype(const ValueType& v) {
70     for (const auto& b : v)
71         if (b != ZeroBlock) return false;
72     return true;
73 }
74
75 // -----
76 // Serialisation compacte CipherText <-> ValueType
77 // (prefixe de longueur réduit a 16 bits, cf. rapport)
78 // -----
79 namespace {
80     void append_u16(std::vector<unsigned char>& out, std::uint16_t v) {
81         out.push_back(static_cast<unsigned char>((v >> 8) & 0xff));
82         out.push_back(static_cast<unsigned char>(v & 0xff));
83     }
84
85     std::uint16_t read_u16(const std::vector<unsigned char>& data, size_t& offset) {
86         if (offset + 2 > data.size())
87             throw std::runtime_error("Invalid serialized payload length");
88         std::uint16_t value = 0;
89         value |= static_cast<std::uint16_t>(data[offset++]) << 8;
90         value |= static_cast<std::uint16_t>(data[offset++]);
91         return value;
92     }
93
94     std::vector<unsigned char> encode_mpz(const BICYCL::Mpz& value) {
95         const int sign = value.sgn();
96         const size_t nbits = value.nbits();
97         const size_t nbytes = (nbits + 7u) / 8u;
98         std::vector<unsigned char> out;
99         out.reserve(1 + 2 + nbytes);

```



```

100
101     out.push_back(sign < 0 ? 0x01 : (sign > 0 ? 0x00 : 0x02));
102     append_u16(out, static_cast<std::uint16_t>(nbytes));
103
104     if (sign != 0) {
105         std::vector<unsigned char> abs_bytes(nbytes);
106         mpz_export(abs_bytes.data(), nullptr, 1, 1, 0, 0,
107             static_cast<mpz_srcptr>(value));
108         out.insert(out.end(), abs_bytes.begin(), abs_bytes.end());
109     }
110     return out;
111 }
112
113 BICYCL::Mpz decode_mpz(const std::vector<unsigned char>& data, size_t& offset) {
114     const unsigned char marker = data[offset++];
115     const std::uint16_t len = read_u16(data, offset);
116     std::vector<unsigned char> bytes(data.begin() + offset, data.begin() + offset
117         + len);
118     offset += len;
119     if (marker == 0x02) return BICYCL::Mpz(0ul);
120     BICYCL::Mpz value(bytes);
121     if (marker == 0x01) value.neg();
122     return value;
123 }
124
125 std::vector<unsigned char> encode_qfi(const BICYCL::QFI& qfi) {
126     std::vector<unsigned char> out;
127     for (auto& part : {encode_mpz(qfi.a()), encode_mpz(qfi.b()),
128         encode_mpz(qfi.c())})
129         out.insert(out.end(), part.begin(), part.end());
130     return out;
131 }
132
133 BICYCL::QFI decode_qfi(const std::vector<unsigned char>& data, size_t& offset) {
134     BICYCL::Mpz a = decode_mpz(data, offset);
135     BICYCL::Mpz b = decode_mpz(data, offset);
136     BICYCL::Mpz c = decode_mpz(data, offset);
137     return BICYCL::QFI(a, b, c);
138 }
139
140 void ciphertext_to_valuetype(const BICYCL::CL_HSMqk::CipherText& ct, ValueType& out) {
141     std::vector<unsigned char> payload;
142     for (auto& part : {encode_qfi(ct.c1()), encode_qfi(ct.c2())})
143         payload.insert(payload.end(), part.begin(), part.end());
144
145     std::fill(out.begin(), out.end(), ZeroBlock);
146     std::uint16_t payload_len = static_cast<std::uint16_t>(payload.size());
147     if (sizeof(payload_len) + payload.size() > kMaxValueBytes)
148         throw std::runtime_error("Serialized ciphertext exceeds ValueType capacity");
149
150     std::memcpy(out.data(), &payload_len, sizeof(payload_len));
151     std::memcpy(reinterpret_cast<unsigned char*>(out.data()) + sizeof(payload_len),
152         payload.data(), payload.size());
153 }
154
155 BICYCL::CL_HSMqk::CipherText valuetype_to_ciphertext(
156     const BICYCL::CL_HSMqk& crypto_system, const ValueType& vt) {
157     const unsigned char* raw = reinterpret_cast<const unsigned char*>(vt.data());
158     std::uint16_t payload_len = 0;
159     std::memcpy(&payload_len, raw, sizeof(payload_len));
160     std::vector<unsigned char> payload(raw + sizeof(payload_len),
161         raw + sizeof(payload_len) + payload_len);
162
163     size_t offset = 0;
164     BICYCL::QFI c1 = decode_qfi(payload, offset);
165     BICYCL::QFI c2 = decode_qfi(payload, offset);
166     return BICYCL::CL_HSMqk::CipherText(c1, c2);
167 }
168
169 // -----
170 // Main
171 // -----
172 int main() {

```

```

171     try {
172         // 1. Initialisation de BICYCL : q est désormais un nombre premier
173         //       de 256 bits (dimensionnement réaliste, sécurité 128 bits)
174         BICYCL::RandGen randgen;
175         BICYCL::Mpz q;
176         q = std::string("57896044618658097711785492504343953926634992332820"
177                        "282019728792003956564820063");
178
179         BICYCL::CL_HSMqk crypto_system(q, 1, 1024, randgen);
180         auto secret_key = crypto_system.keygen(randgen);
181         auto public_key = crypto_system.keygen(secret_key);
182
183         // 2. Paramètres Paxos (n éléments, tolérance delta)
184         u64 n = 8, m = 5; i64 delta = 2;
185         u64 N = (2 * delta + 1) * n;
186         block seed = toBlock(0, 0);
187         PRNG prng(seed);
188
189         std::vector<block> serverSet1(n), serverSet2(n);
190         std::vector<block> keys1(N), keys2(N);
191         std::vector<ValueType> vals1(N), vals2(N);
192
193         for (u64 i = 0; i < n; ++i) {
194             serverSet1[i] = prng.get<block>();
195             serverSet2[i] = prng.get<block>();
196         }
197
198         // 3. Chiffrement des parts partagées (share1 + share2 = 0 mod q)
199         for (u64 i = 0; i < n; ++i) {
200             block clear_share1 = prng.get<block>();
201             BICYCL::Mpz m1_mpz(clear_share1.get<u64>(0) % 65537UL);
202             BICYCL::Mpz m2_mpz;
203             if (m1_mpz == BICYCL::Mpz(0ul)) m2_mpz = BICYCL::Mpz(0ul);
204             else BICYCL::Mpz::sub(m2_mpz, q, m1_mpz);
205
206             BICYCL::CL_HSMqk::ClearText m1(crypto_system, m1_mpz);
207             BICYCL::CL_HSMqk::ClearText m2(crypto_system, m2_mpz);
208             auto cipher1 = crypto_system.encrypt(public_key, m1, randgen);
209             auto cipher2 = crypto_system.encrypt(public_key, m2, randgen);
210
211             ValueType share1_encoded, share2_encoded;
212             ciphertext_to_valuetype(cipher1, share1_encoded);
213             ciphertext_to_valuetype(cipher2, share2_encoded);
214
215             u64 x1 = serverSet1[i].get<u64>(0), x2 = serverSet2[i].get<u64>(0);
216             for (i64 j = 0; j < 2 * delta + 1; ++j) {
217                 i64 offset = -delta + j;
218                 keys1[(2*delta+1)*i + j] = toBlock(serverSet1[i].get<u64>(1),
219                    static_cast<u64>(static_cast<i64>(x1) + offset));
220                 keys2[(2*delta+1)*i + j] = toBlock(serverSet2[i].get<u64>(1),
221                    static_cast<u64>(static_cast<i64>(x2) + offset));
222                 vals1[(2*delta+1)*i + j] = share1_encoded;
223                 vals2[(2*delta+1)*i + j] = share2_encoded;
224             }
225         }
226
227         // 4. Encodage Paxos OKVS
228         volePSI::Paxos<u64> paxos;
229         paxos.init(N, 3, 40, volePSI::PaxosParam::DenseType::Binary, seed);
230         std::vector<ValueType> storage1(paxos.size()), storage2(paxos.size());
231         paxos.solve<ValueType>(keys2, vals2, storage2, &prng);
232         paxos.solve<ValueType>(keys1, vals1, storage1, &prng);
233         std::cout << "Paxos OKVS encoded successfully\n";
234
235         // 5. Decodage : 2 clés synchronisées, 1 désynchronisée, 2 aléatoires
236         std::vector<block> clientSet1(m), clientSet2(m);
237         std::vector<ValueType> decoded1(m), decoded2(m);
238         clientSet1[0] = keys1[0]; clientSet2[0] = keys2[0];
239         clientSet1[1] = keys1[5]; clientSet2[1] = keys2[5];
240         clientSet1[2] = keys1[10]; clientSet2[2] = keys2[15];
241         for (u64 i = 3; i < m; ++i) {
242             clientSet1[i] = prng.get<block>();
243             clientSet2[i] = prng.get<block>();

```

```

244     }
245
246     auto t0 = std::chrono::high_resolution_clock::now();
247     paxos.decode<ValueType>(clientSet1, decoded1, storage1);
248     paxos.decode<ValueType>(clientSet2, decoded2, storage2);
249     auto t1 = std::chrono::high_resolution_clock::now();
250     double decodeMs = std::chrono::duration_cast<std::chrono::microseconds>(
251         t1 - t0).count() / 1000.0;
252     std::cout << "decode time = " << decodeMs << " ms\n";
253
254     // 6. Reconstruction homomorphe et verification (Add puis Decrypt == 0)
255     for (u64 i = 0; i < m; ++i) {
256         bool found = false;
257         try {
258             auto c1 = valuetype_to_ciphertext(crypto_system, decoded1[i]);
259             auto c2 = valuetype_to_ciphertext(crypto_system, decoded2[i]);
260             auto c_sum = crypto_system.add_ciphertexts(public_key, c1, c2,
261                 randgen);
262             auto decrypted_sum = crypto_system.decrypt(secret_key, c_sum);
263             found = (decrypted_sum == BICYCL::Mpz(0ul));
264         } catch (const std::exception&) { found = false; }
265
266         std::cout << (found
267             ? "decode succeeded: The server already has the number "
268             : "decode failed: The server does not have the number value ")
269             << i << "\n";
270     }
271     std::cout << "decode success" << std::endl;
272
273     } catch (const std::exception& e) {
274         std::cerr << "Error: " << e.what() << std::endl;
275         return 1;
276     }
277     return 0;

```

Listing 6 – FPSUServer_bicycl.cpp – version finale

Le code source complet, commenté, ainsi que l'historique de développement (y compris le pont *bridge.go* de SimplePIR), sont disponibles dans le dépôt Git du projet : <https://github.com/yasserito111/CRYPTO>.